

What is a Vector in Machine Learning?

Visual Explanation for YouTube Shorts

Vector = Ordered list of numbers representing data

- Foundation of all ML algorithms
- Data as mathematical objects
- Enable computation and comparison

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import seaborn as sns

plt.style.use('dark_background')
sns.set_palette("bright")

# Set random seed for reproducibility
np.random.seed(42)
```

1. What is a Vector?

Vector = List of numbers with direction and magnitude

```
In [2]: # Simple 2D vector
vector_2d = np.array([3, 4])
print(f"2D Vector: {vector_2d}")
print(f"Shape: {vector_2d.shape}")
print(f"Dimensions: {vector_2d.ndim}D")

# 3D vector
vector_3d = np.array([2, 3, 6])
print(f"\n3D Vector: {vector_3d}")

# Higher dimensional vector (typical in ML)
vector_high_dim = np.array([1.2, 3.4, 5.6, 2.1, 4.8])
print(f"\n5D Vector: {vector_high_dim}")
print(f"This could represent: [age, salary, years_exp, rating, hours_worked]")
```

```
2D Vector: [3 4]
Shape: (2,)
Dimensions: 1D
```

```
3D Vector: [2 3 6]
```

```
5D Vector: [1.2 3.4 5.6 2.1 4.8]
This could represent: [age, salary, years_exp, rating, hours_worked]
```

2. Visualizing 2D Vectors

See vectors as arrows in space!

```
In [3]: # Create multiple 2D vectors
vectors = {
    'v1': np.array([3, 4]),
    'v2': np.array([-2, 3]),
    'v3': np.array([4, -2]),
    'v4': np.array([-3, -3])
}

colors = ['cyan', 'magenta', 'yellow', 'lime']

plt.figure(figsize=(10, 10))
ax = plt.gca()

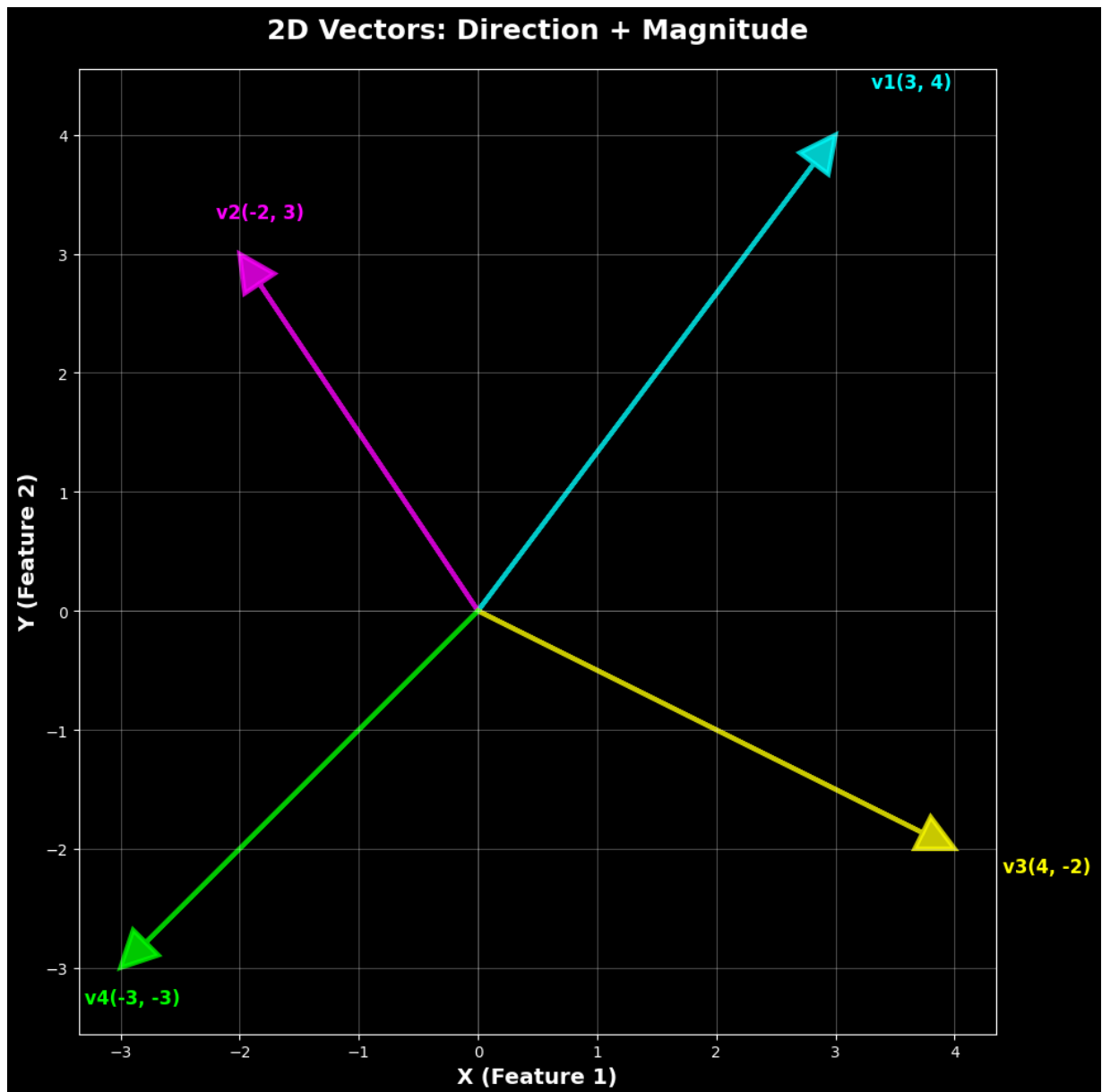
# Plot each vector
for (name, vec), color in zip(vectors.items(), colors):
    # Arrow from origin to vector point
    plt.arrow(0, 0, vec[0], vec[1],
              head_width=0.3, head_length=0.3,
              fc=color, ec=color, linewidth=3,
              length_includes_head=True, alpha=0.8)

    # Label
    plt.text(vec[0]*1.1, vec[1]*1.1, f'{name}{tuple(vec)}',
             fontsize=12, fontweight='bold', color=color)

    # Calculate magnitude
    magnitude = np.linalg.norm(vec)
    print(f'{name}: {vec} -> Magnitude: {magnitude:.2f}')

# Set equal aspect ratio and grid
plt.axhline(y=0, color='white', linewidth=0.5, alpha=0.3)
plt.axvline(x=0, color='white', linewidth=0.5, alpha=0.3)
plt.grid(True, alpha=0.3)
plt.xlim(-5, 5)
plt.ylim(-5, 5)
plt.xlabel('X (Feature 1)', fontsize=14, fontweight='bold')
plt.ylabel('Y (Feature 2)', fontsize=14, fontweight='bold')
plt.title('2D Vectors: Direction + Magnitude', fontsize=18, fontweight='bold')
plt.axis('equal')
plt.tight_layout()
plt.show()
```

```
v1: [3 4] -> Magnitude: 5.00
v2: [-2 3] -> Magnitude: 3.61
v3: [ 4 -2] -> Magnitude: 4.47
v4: [-3 -3] -> Magnitude: 4.24
```



3. Vector Operations

Addition and Scalar Multiplication

```
In [4]: # Define two vectors
v1 = np.array([2, 3])
v2 = np.array([3, 1])

# Vector addition
v_sum = v1 + v2

# Scalar multiplication
v_scaled = 1.5 * v1

print(f"v1 = {v1}")
print(f"v2 = {v2}")
print(f"v1 + v2 = {v_sum}")
```

```

print(f"1.5 * v1 = {v_scaled}")

# Visualize operations
fig, axes = plt.subplots(1, 2, figsize=(16, 7))

# Plot 1: Vector Addition
ax1 = axes[0]
ax1.arrow(0, 0, v1[0], v1[1], head_width=0.3, head_length=0.3,
          fc='cyan', ec='cyan', linewidth=3, label='v1', alpha=0.8)
ax1.arrow(0, 0, v2[0], v2[1], head_width=0.3, head_length=0.3,
          fc='magenta', ec='magenta', linewidth=3, label='v2', alpha=0.8)
ax1.arrow(v1[0], v1[1], v2[0], v2[1], head_width=0.3, head_length=0.3,
          fc='magenta', ec='magenta', linewidth=2, linestyle='--', alpha=0.5)
ax1.arrow(0, 0, v_sum[0], v_sum[1], head_width=0.4, head_length=0.4,
          fc='yellow', ec='yellow', linewidth=4, label='v1 + v2', alpha=0.9)

ax1.axhline(y=0, color='white', linewidth=0.5, alpha=0.3)
ax1.axvline(x=0, color='white', linewidth=0.5, alpha=0.3)
ax1.grid(True, alpha=0.3)
ax1.set_xlim(-1, 7)
ax1.set_ylim(-1, 6)
ax1.set_xlabel('X', fontsize=14, fontweight='bold')
ax1.set_ylabel('Y', fontsize=14, fontweight='bold')
ax1.set_title('Vector Addition: v1 + v2', fontsize=16, fontweight='bold', pad=5)
ax1.legend(fontsize=12, loc='upper left')
ax1.set_aspect('equal')

# Plot 2: Scalar Multiplication
ax2 = axes[1]
ax2.arrow(0, 0, v1[0], v1[1], head_width=0.3, head_length=0.3,
          fc='cyan', ec='cyan', linewidth=3, label='v1 (original)', alpha=0.8)
ax2.arrow(0, 0, v_scaled[0], v_scaled[1], head_width=0.3, head_length=0.3,
          fc='yellow', ec='yellow', linewidth=4, label='1.5 * v1 (scaled)', alpha=0.9)
ax2.arrow(0, 0, 0.5*v1[0], 0.5*v1[1], head_width=0.2, head_length=0.2,
          fc='magenta', ec='magenta', linewidth=3, label='0.5 * v1 (shrunk)', alpha=0.8)

ax2.axhline(y=0, color='white', linewidth=0.5, alpha=0.3)
ax2.axvline(x=0, color='white', linewidth=0.5, alpha=0.3)
ax2.grid(True, alpha=0.3)
ax2.set_xlim(-1, 5)
ax2.set_ylim(-1, 6)
ax2.set_xlabel('X', fontsize=14, fontweight='bold')
ax2.set_ylabel('Y', fontsize=14, fontweight='bold')
ax2.set_title('Scalar Multiplication: k * v1', fontsize=16, fontweight='bold', pad=5)
ax2.legend(fontsize=12, loc='upper left')
ax2.set_aspect('equal')

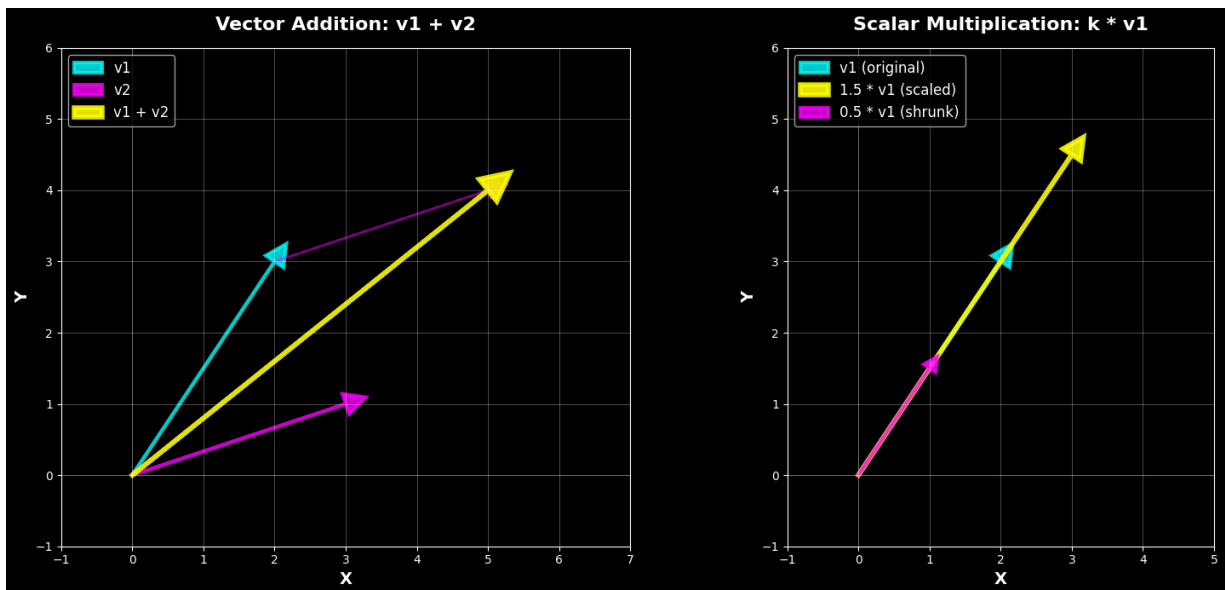
plt.tight_layout()
plt.show()

```

```

v1 = [2 3]
v2 = [3 1]
v1 + v2 = [5 4]
1.5 * v1 = [3.  4.5]

```



4. Vectors in Machine Learning

Real-world examples

```
In [5]: # Example 1: House features as vector
house = np.array([1500, 3, 2, 15]) # [sqft, bedrooms, bathrooms, age]
print("House Vector (Feature Vector):")
print(f" Square Feet: {house[0]}")
print(f" Bedrooms: {house[1]}")
print(f" Bathrooms: {house[2]}")
print(f" Age (years): {house[3]}")
print(f" Vector: {house}\n")

# Example 2: User profile as vector
user = np.array([25, 50000, 3.5, 120]) # [age, income, rating, hours_online]
print("User Vector:")
print(f" Age: {user[0]}")
print(f" Income: ${user[1]}")
print(f" Rating: {user[2]}")
print(f" Hours Online/month: {user[3]}")
print(f" Vector: {user}\n")

# Example 3: Image as vector (flattened)
tiny_image = np.random.randint(0, 256, size=(3, 3)) # 3x3 grayscale image
image_vector = tiny_image.flatten()
print("Image as Matrix:")
print(tiny_image)
print(f"\nImage as Vector (flattened): {image_vector}")
print(f"Vector length: {len(image_vector)} (3x3 = 9 pixels)")
```

House Vector (Feature Vector):
Square Feet: 1500
Bedrooms: 3
Bathrooms: 2
Age (years): 15
Vector: [1500 3 2 15]

User Vector:
Age: 25.0
Income: \$50000.0
Rating: 3.5
Hours Online/month: 120.0
Vector: [2.5e+01 5.0e+04 3.5e+00 1.2e+02]

Image as Matrix:
[[102 179 92]
[14 106 71]
[188 20 102]]

Image as Vector (flattened): [102 179 92 14 106 71 188 20 102]
Vector length: 9 (3x3 = 9 pixels)

5. Vector Similarity and Distance

How "close" are two vectors?

```
In [6]: # Create sample data points (users with 2 features)
user_a = np.array([3, 4]) # User A: [hours_study, test_score]
user_b = np.array([6, 8]) # User B: similar pattern
user_c = np.array([1, 7]) # User C: different pattern

# Calculate distances
from numpy.linalg import norm

# Euclidean distance
dist_ab = norm(user_a - user_b)
dist_ac = norm(user_a - user_c)
dist_bc = norm(user_b - user_c)

# Cosine similarity
def cosine_similarity(v1, v2):
    return np.dot(v1, v2) / (norm(v1) * norm(v2))

cos_ab = cosine_similarity(user_a, user_b)
cos_ac = cosine_similarity(user_a, user_c)

print("Distance Metrics:")
print(f"Distance A-B: {dist_ab:.2f}")
print(f"Distance A-C: {dist_ac:.2f}")
print(f"Distance B-C: {dist_bc:.2f}")
print(f"\nCosine Similarity A-B: {cos_ab:.3f} (1 = identical direction)")
print(f"Cosine Similarity A-C: {cos_ac:.3f}")

# Visualize
```

```

plt.figure(figsize=(12, 10))

# Plot users as vectors
users = {'A': user_a, 'B': user_b, 'C': user_c}
colors_map = {'A': 'cyan', 'B': 'lime', 'C': 'magenta'}

for name, vec in users.items():
    color = colors_map[name]
    plt.arrow(0, 0, vec[0], vec[1],
              head_width=0.4, head_length=0.4,
              fc=color, ec=color, linewidth=4, alpha=0.8)
    plt.scatter(vec[0], vec[1], s=400, c=color,
                edgecolors='white', linewidth=3, zorder=5)
    plt.text(vec[0]*1.15, vec[1]*1.15, f'User {name}\n{tuple(vec)}',
             fontsize=13, fontweight='bold', color=color,
             ha='center', va='center')

# Draw distance lines
plt.plot([user_a[0], user_b[0]], [user_a[1], user_b[1]],
         'yellow', linewidth=2, linestyle='--', alpha=0.6, label=f'Dist A-B: ')
plt.plot([user_a[0], user_c[0]], [user_a[1], user_c[1]],
         'orange', linewidth=2, linestyle='--', alpha=0.6, label=f'Dist A-C: ')

plt.axhline(y=0, color='white', linewidth=0.5, alpha=0.3)
plt.axvline(x=0, color='white', linewidth=0.5, alpha=0.3)
plt.grid(True, alpha=0.3)
plt.xlim(-1, 8)
plt.ylim(-1, 10)
plt.xlabel('Hours Studied', fontsize=14, fontweight='bold')
plt.ylabel('Test Score', fontsize=14, fontweight='bold')
plt.title('Vector Similarity: Which users are more similar?',
          fontsize=18, fontweight='bold', pad=20)
plt.legend(fontsize=12, loc='upper left')
plt.tight_layout()
plt.show()

print("\n💡 Insight: Users A and B are more similar (smaller distance, higher")

```

Distance Metrics:

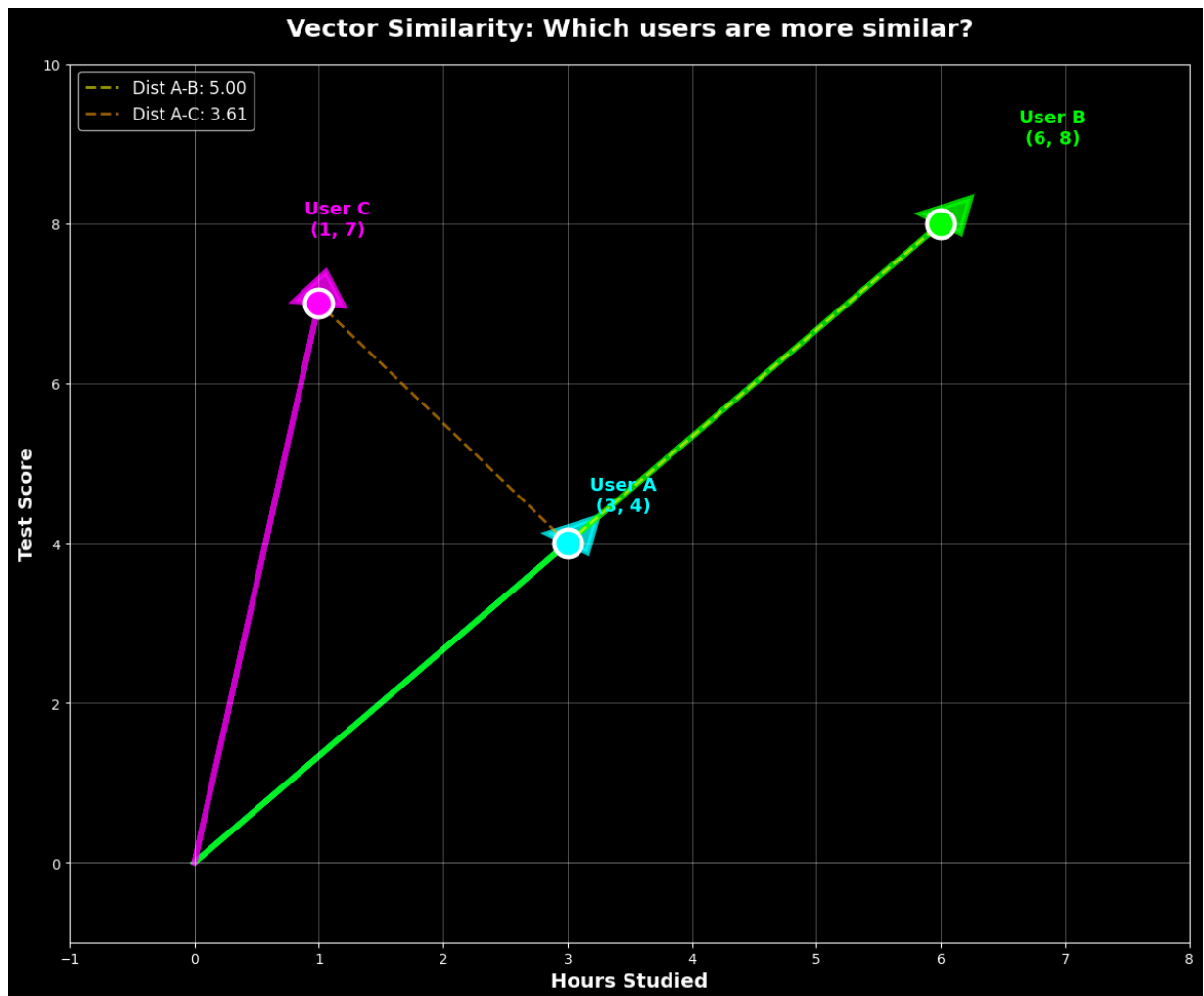
Distance A-B: 5.00

Distance A-C: 3.61

Distance B-C: 5.10

Cosine Similarity A-B: 1.000 (1 = identical direction)

Cosine Similarity A-C: 0.877



💡 Insight: Users A and B are more similar (smaller distance, higher cosine similarity)

6. 3D Vector Visualization

Vectors in 3D space

```
In [7]: # Create 3D vectors
v1_3d = np.array([3, 2, 5])
v2_3d = np.array([1, 4, 2])
v_sum_3d = v1_3d + v2_3d

fig = plt.figure(figsize=(14, 10))
ax = fig.add_subplot(111, projection='3d')

# Plot vectors from origin
origin = np.array([0, 0, 0])

# Vector 1
ax.quiver(0, 0, 0, v1_3d[0], v1_3d[1], v1_3d[2],
          color='cyan', arrow_length_ratio=0.15, linewidth=4, alpha=0.8, label='v1')

# Vector 2
ax.quiver(0, 0, 0, v2_3d[0], v2_3d[1], v2_3d[2],
          color='magenta', arrow_length_ratio=0.15, linewidth=4, alpha=0.8, label='v2')
```

```

        color='magenta', arrow_length_ratio=0.15, linewidth=4, alpha=0.8, l

# Sum vector
ax.quiver(0, 0, 0, v_sum_3d[0], v_sum_3d[1], v_sum_3d[2],
        color='yellow', arrow_length_ratio=0.15, linewidth=5, alpha=0.9, la

# Vector from v1 to sum (parallel to v2)
ax.quiver(v1_3d[0], v1_3d[1], v1_3d[2], v2_3d[0], v2_3d[1], v2_3d[2],
        color='magenta', arrow_length_ratio=0.15, linewidth=2,
        alpha=0.4, linestyle='dashed')

# Plot points
ax.scatter(*v1_3d, s=300, c='cyan', edgecolors='white', linewidth=2, alpha=0
ax.scatter(*v2_3d, s=300, c='magenta', edgecolors='white', linewidth=2, alph
ax.scatter(*v_sum_3d, s=400, c='yellow', edgecolors='white', linewidth=3, al

# Labels
ax.text(v1_3d[0], v1_3d[1], v1_3d[2], f' v1{tuple(v1_3d)}',
        fontsize=11, color='cyan', fontweight='bold')
ax.text(v2_3d[0], v2_3d[1], v2_3d[2], f' v2{tuple(v2_3d)}',
        fontsize=11, color='magenta', fontweight='bold')
ax.text(v_sum_3d[0], v_sum_3d[1], v_sum_3d[2], f' sum{tuple(v_sum_3d)}',
        fontsize=11, color='yellow', fontweight='bold')

# Styling
ax.set_xlabel('X', fontsize=12, fontweight='bold')
ax.set_ylabel('Y', fontsize=12, fontweight='bold')
ax.set_zlabel('Z', fontsize=12, fontweight='bold')
ax.set_title('3D Vector Addition', fontsize=18, fontweight='bold', pad=20)
ax.legend(fontsize=12, loc='upper left')

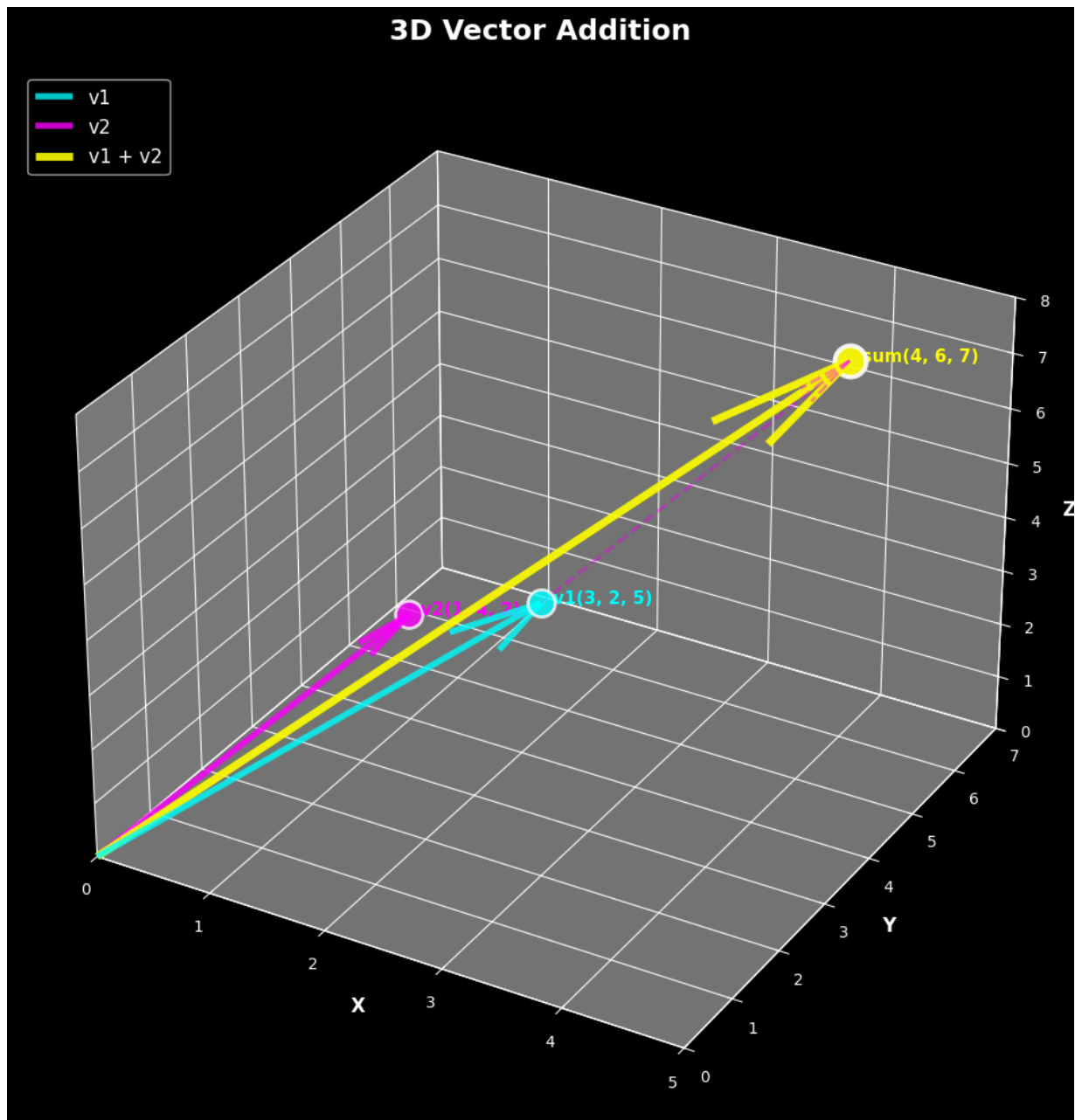
# Set limits
ax.set_xlim([0, 5])
ax.set_ylim([0, 7])
ax.set_zlim([0, 8])

# Add grid
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print(f"v1 = {v1_3d}")
print(f"v2 = {v2_3d}")
print(f"v1 + v2 = {v_sum_3d}")
print(f"\nMagnitude of v1: {norm(v1_3d):.2f}")
print(f"Magnitude of v2: {norm(v2_3d):.2f}")
print(f"Magnitude of sum: {norm(v_sum_3d):.2f}")

```



$v1 = [3 \ 2 \ 5]$
 $v2 = [1 \ 4 \ 2]$
 $v1 + v2 = [4 \ 6 \ 7]$

Magnitude of $v1$: 6.16
Magnitude of $v2$: 4.58
Magnitude of sum : 10.05

7. Key Takeaways

What is a Vector?

- Ordered list of numbers
- Has magnitude (length) and direction
- Foundation of ML: data $\rightarrow$ vectors

Vector Operations:

- Addition: Combine vectors
- Scalar multiplication: Scale vectors
- Dot product: Measure similarity

In Machine Learning:

- **Feature vectors:** Represent data points (houses, users, images)
- **Word embeddings:** Words as vectors (Word2Vec, BERT)
- **Neural networks:** Inputs/outputs are vectors
- **Similarity:** Compare vectors using distance/cosine

Common Applications:

- Recommendation systems (user/item vectors)
- Image classification (pixel vectors)
- NLP (word/sentence embeddings)
- Clustering (grouping similar vectors)

Remember:

- Everything in ML is a vector!
- Distance = how different
- Similarity = how alike
- Math enables ML magic! ✨